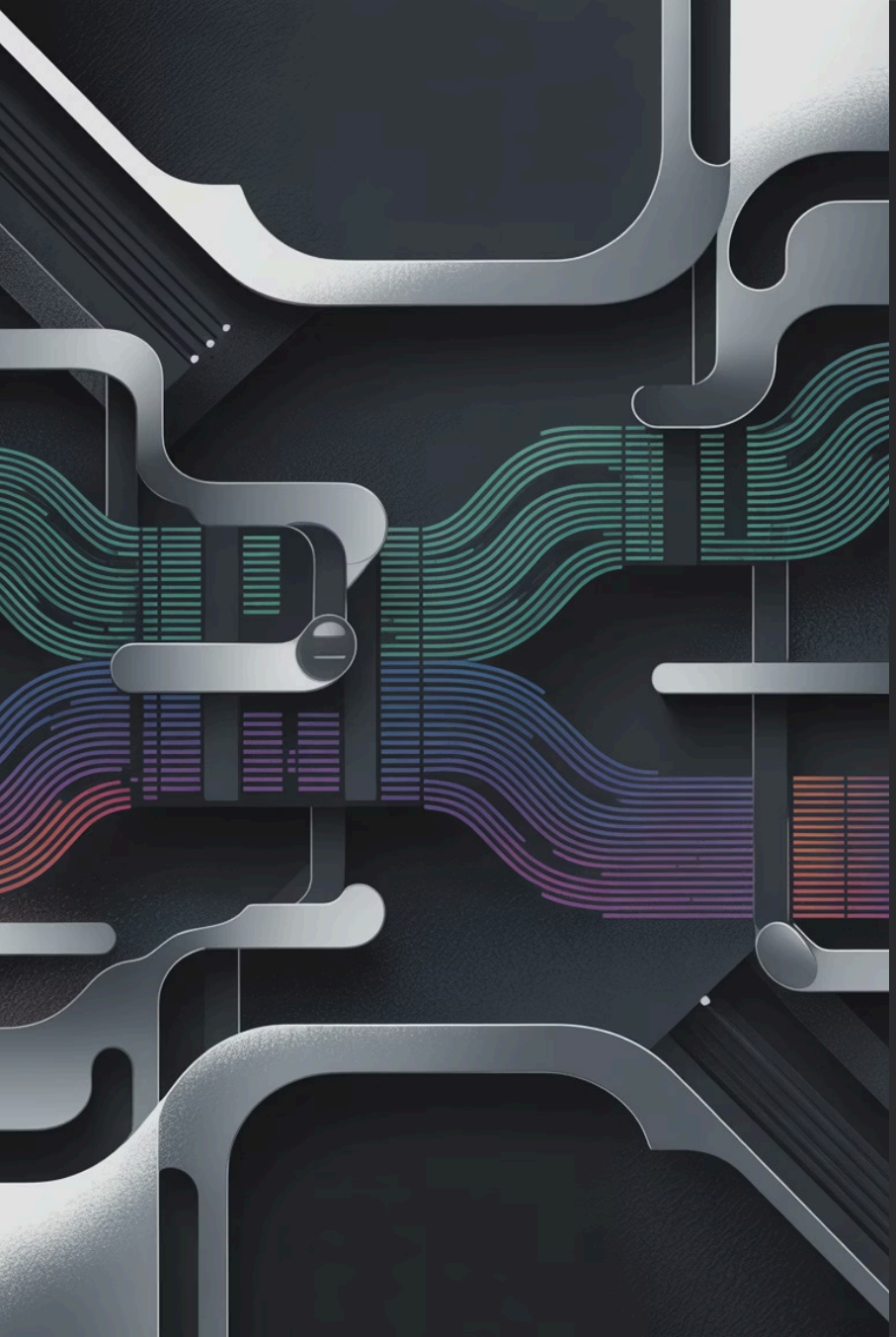


Module 07 - Data Engineering for LLM Pipelines

Building the Foundation for AI Engineering Excellence

Module Learning Journey

01

GroupBy Operations

Aggregate data by categories using groupby() and multiple aggregation functions

02

Join Strategies

Understand inner, left, outer, and right joins with practical merge patterns

03

Real-World Application

Apply these techniques to customer and order data in hands-on lab exercises

Why GroupBy and Joins Matter in AI Pipelines

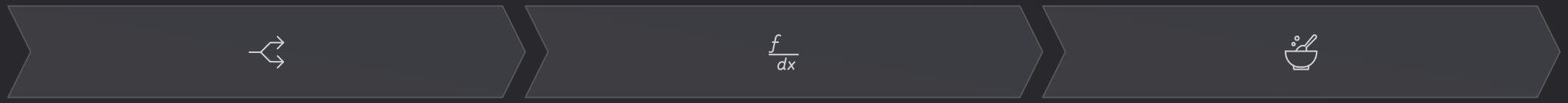
In data engineering for LLMs, you'll frequently need to aggregate information from different sources and combine datasets. Whether you're preparing training data, enriching prompts with context, or analyzing model outputs, these operations are foundational.

GroupBy operations let you calculate statistics by category—essential for feature engineering. Joins allow you to enrich datasets by combining related information from multiple tables or data sources.

Part 1: GroupBy Operations

Understanding GroupBy: The Concept

GroupBy is a split-apply-combine operation. Think of it as organizing your data into meaningful groups, performing calculations on each group, and then combining the results back together.



Split

Divide data into groups based on one or more columns

Apply

Calculate aggregations like mean, sum, count for each group

Combine

Merge results into a new DataFrame with aggregated values

Basic GroupBy Syntax

The fundamental pattern for groupby operations in Pandas follows this structure. You specify which column(s) to group by, then apply aggregation functions to calculate summary statistics.

```
# Group by a single column
df.groupby('category')['value'].mean()

# Group by multiple columns
df.groupby(['region', 'category'])['sales'].sum()

# Using agg() for multiple aggregations
df.groupby('category').agg({'price': 'mean', 'quantity': 'sum'})
```

Common Aggregation Functions

Pandas provides a rich set of aggregation functions that you can apply to grouped data. These functions help you extract meaningful insights from your datasets.

Statistical Functions

- `mean()` - Average value
- `median()` - Middle value
- `std()` - Standard deviation
- `min()`, `max()` - Extremes

Counting Functions

- `count()` - Number of values
- `nunique()` - Unique values
- `size()` - Group size

Summary Functions

- `sum()` - Total
- `first()`, `last()` - Endpoints
- `describe()` - Full summary

Single Aggregation Example

Let's start with a simple example calculating average order value by customer. This is a common pattern when analyzing customer behavior or preparing features for ML models.

```
# Calculate mean order value
# per customer
df.groupby('customer_id')['order_total'].mean()

# With column rename
df.groupby('customer_id').agg(
    avg_order_value=('order_total', 'mean')
)
```

Multiple Aggregations with agg()

The `agg()` method unlocks powerful capabilities by allowing you to apply different aggregation functions to different columns simultaneously. This is especially useful when you need various statistics in a single operation.

```
# Apply different aggregations to different columns
customer_summary = df.groupby('customer_id').agg({
 'order_total': 'mean', # Average order value
 'order_id': 'count', # Number of orders
 'quantity': 'sum', # Total items purchased
 'order_date': 'max' # Most recent order
})

# Rename columns for clarity
customer_summary.columns = ['avg_order', 'num_orders', 'total_items', 'last_order']
```

Advanced: Multiple Functions per Column

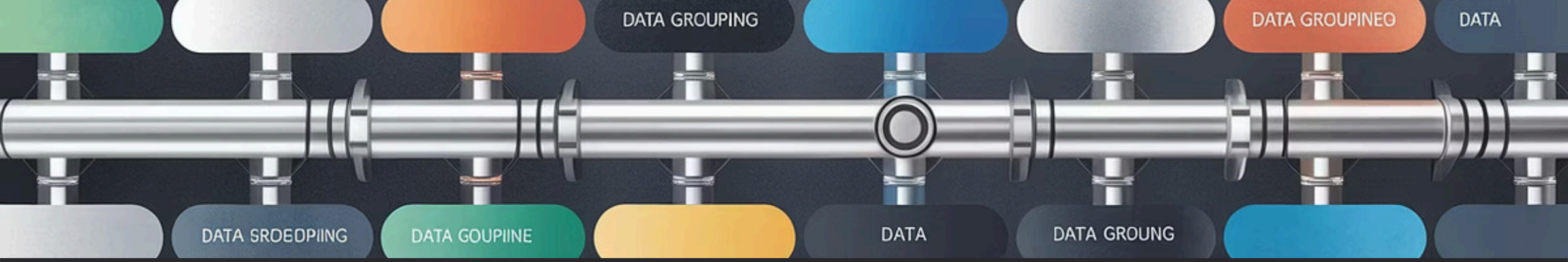
You can apply multiple aggregation functions to the same column, creating a MultiIndex DataFrame. This is powerful for comprehensive statistical summaries.

```
# Multiple aggregations per column
result = df.groupby('product_category').agg({     # assign to `result`
    'price': ['mean', 'min', 'max', 'std'],
    'quantity': ['sum', 'count'],
})

# Flatten MultiIndex columns
result.columns = ['_'.join(col) for col in result.columns]
```

Result columns:

- price_mean
- price_min
- price_max
- price_std
- quantity_sum
- quantity_count



GroupBy in LLM Data Pipelines

When preparing data for LLMs, groupby operations help you aggregate information into meaningful contexts. For example, you might group customer interactions by user_id to create conversation histories, or group documents by topic to build domain-specific training sets.

Practical Lab Scenario: Customer Orders

In the next lab, you'll work with two datasets: customers and orders. You'll use groupby to calculate metrics like total spending per customer, average order value, and purchase frequency—all essential for customer segmentation and personalization in AI applications.

Customers Dataset

Contains customer_id, name, email, and registration date

Orders Dataset

Contains order_id, customer_id, order_date, and order_total

Goal

Aggregate order metrics and join with customer info for analysis

Part 2: Joining DataFrames

Why We Need Joins

Real-world data is rarely contained in a single table. Customer information lives in one table, orders in another, products in a third. Joins allow you to combine related data from multiple sources based on common keys.

In AI engineering, you'll frequently join datasets to enrich your training data, add contextual information to prompts, or combine predictions with original data for analysis.

The Four Join Types

Inner Join

Returns only rows where the key exists in **both** DataFrames. Most restrictive but ensures complete data.

Left Join

Returns all rows from the left DataFrame, with matching rows from the right. Preserves left table completely.

Right Join

Returns all rows from the right DataFrame, with matching rows from the left. Preserves right table completely.

Outer Join

Returns all rows from both DataFrames, filling missing values with NaN. Most inclusive, captures everything.

Inner Join: The Intersection

An inner join returns only the rows where the join key exists in both DataFrames. Think of it as the intersection of two sets.

Use inner join when:

- You only want complete records
- Missing matches indicate data quality issues
- You need to ensure both sides have values

```
pd.merge(customers, orders,
        on='customer_id',
        how='inner')
```

If a customer has no orders, they won't appear in the result. If an order has an invalid customer_id, it's excluded.

Left Join: Keep All Left Records

A left join preserves all rows from the left DataFrame and adds matching data from the right. Non-matching rows from the right are filled with NaN values.

```
# Keep all customers, add order info where available
pd.merge(customers, order_summary,
        on='customer_id',
        how='left')
```

Example Result: All 100 customers appear in output. Customers without orders show NaN for order-related columns.

Use left join when:

- The left DataFrame is your primary dataset
- You want to enrich left records with optional right data
- It's okay if some left records have no matches

Outer Join: Keep Everything

An outer join (also called a full outer join) returns all rows from both DataFrames, filling in NaN wherever data is missing. This is the most inclusive join type.

Complete Data Preservation

No information is lost—every row from both DataFrames appears in the result

Data Quality Checks

Useful for identifying orphaned records or mismatches between datasets

Exploratory Analysis

Helps discover unexpected patterns or missing relationships in your data

```
pd.merge(df1, df2, on='key', how='outer')
```

Choosing the Right Join Type

The join type you choose has significant implications for your analysis and downstream AI pipelines. Here's a decision framework:

1

Start with Inner Join

Default to inner join when you need clean, complete records with guaranteed data on both sides

2

Use Left Join for Enrichment

When you have a primary dataset and want to add supplementary information without losing records

3

Use Outer Join for Investigation

When exploring data quality issues or needing to see all records from both sources

4

Validate Results

Always check row counts before and after joins to ensure the operation behaved as expected

Join Syntax in Pandas

Pandas provides the `merge()` function for joining DataFrames. The syntax is straightforward but powerful.

```
# Basic merge syntax
pd.merge(
 left_df,
 right_df,
 on='key_column', # Join key
 how='inner' # Join type
)

# Join on different column names
pd.merge(
 customers,
 orders,
 left_on='id',
 right_on='customer_id',
 how='left'
)
```

Joining on Multiple Keys

Sometimes you need to join on multiple columns to uniquely identify matching rows. This is common when dealing with hierarchical data or time-series datasets.

```
# Join on multiple columns
pd.merge(
    sales,
    targets,
    on=['region', 'product_category', 'year'],
    how='inner'
)

# Or use left_on and right_on with lists
pd.merge(
    df1,
    df2,
    left_on=['col_a', 'col_b'],
    right_on=['col_x', 'col_y'],
    how='left'
)
```

Pro Tip: When joining on multiple keys, ensure the combination of keys uniquely identifies rows to avoid unintended duplicate results.

Handling Duplicate Column Names

When both DataFrames have columns with the same name (other than the join key), Pandas automatically adds suffixes to distinguish them. You can customize these suffixes.

```
# Default behavior adds _x and _y suffixes
pd.merge(customers, orders, on='customer_id')
# Results in: name_x, name_y

# Customize suffixes for clarity
pd.merge(
 customers,
 orders,
 on='customer_id',
 suffixes=('_customer', '_order')
)
# Results in: name_customer, name_order
```

Join Validation: Critical for Data Quality

Always validate your joins to ensure they produce expected results. Unexpected row counts can indicate data quality issues, incorrect keys, or logic errors.

1 Check Row Counts

Compare input and output row counts — but the expected counts depend on **key uniqueness**:

- If the **right key is unique** (one-to-one or many-to-one): an inner join returns ⩽ the left row count, and a **left join preserves the left row count exactly**.
- If the **right key is *not* unique** (one-to-many / many-to-many): matching left rows **multiply** — both inner and left joins can return **more** rows than either input. That's the Cartesian explosion on the next slide.

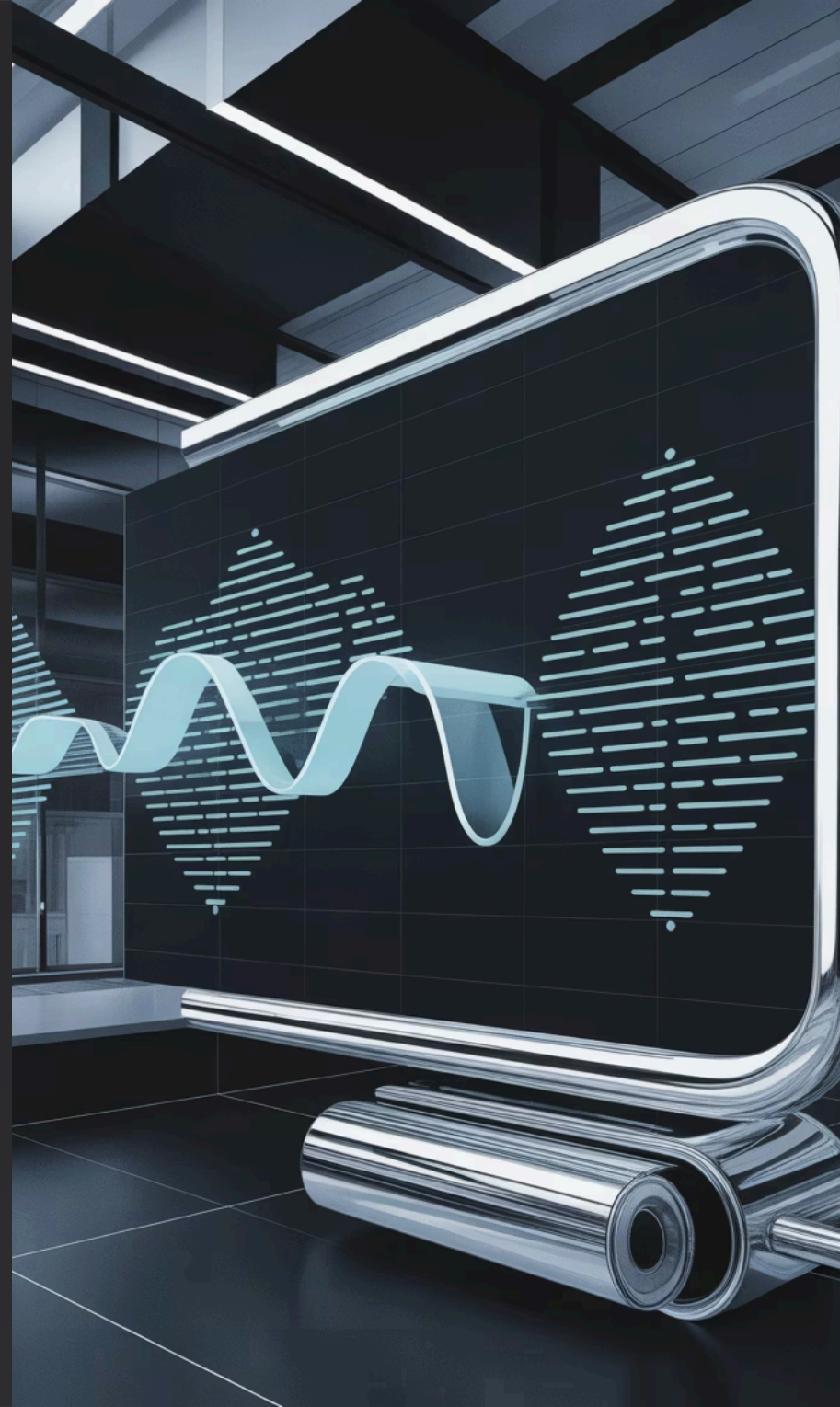
Always check `df['key'].nunique() == len(df)` on the *right* side before trusting a row-count rule.

2 Inspect for NaN Values

Use `df.isna().sum()` to identify missing values introduced by the join, especially in outer and left joins.

3 Verify Key Uniqueness

Check if join keys are unique in both DataFrames using `df['key'].nunique()` vs `len(df)`.



Common Join Pitfalls

Cartesian Explosion

When join keys aren't unique, you get a Cartesian product. A customer with 3 orders joined to 2 matching records creates 6 rows. Always check for unexpected row count increases.

Mixed Data Types

Joining on columns with **different dtypes** (e.g. integer key vs string key) raises a `ValueError` in current pandas — it no longer matches silently. Check with `df.dtypes` and cast to align before merging:

```
orders['customer_id'] = orders['customer_id'].astype(int) # match the key dtype
```

Missing Keys

NaN values in join keys are never matched. A single missing customer_id can cause unexpected results. Clean your data before joining.

Combining GroupBy and Joins

The Powerful Pattern: Aggregate Then Join

One of the most common patterns in data engineering is to aggregate data with groupby, then join the results back to your primary dataset. This enriches your data with calculated features.

```
# Step 1: Aggregate order data
order_summary = orders.groupby('customer_id').agg({
    'order_total': ['sum', 'mean', 'count'],
    'order_date': 'max'
})

# Flatten column names
order_summary.columns = [
    'total_spent', 'avg_order',
    'num_orders', 'last_purchase'
]

# Step 2: Join back to customers
enriched_customers = pd.merge(
    customers,
    order_summary,
    on='customer_id',
    how='left'
)
```

Result: Every customer now has aggregate order metrics attached, ready for analysis or ML feature engineering.

This pattern is essential when building customer 360 views, creating training datasets, or preparing data for LLM context.

Real-World Example: Customer Segmentation

Let's walk through a complete example that demonstrates the power of combining groupby and joins for customer analytics.

```
# Calculate customer lifetime value and behavior metrics
customer_metrics = orders.groupby('customer_id').agg(
    total_revenue=('order_total', 'sum'),
    avg_order_value=('order_total', 'mean'),
    order_count=('order_id', 'count'),
    first_purchase=('order_date', 'min'),
    last_purchase=('order_date', 'max'),
).reset_index()

# "now" that matches the column's timezone: None for naive dates, the tz for
# tz-aware dates. This avoids "Cannot subtract tz-naive and tz-aware" errors.
now = pd.Timestamp.now(tz=customer_metrics['first_purchase'].dt.tz)

customer_metrics['customer_age_days'] = (
    now - customer_metrics['first_purchase']
).dt.days
customer_metrics['days_since_purchase'] = (
    now - customer_metrics['last_purchase']
).dt.days

# Join with customer demographic data
customer_360 = pd.merge(
    customers,
    customer_metrics,
    on='customer_id',
    how='left',
)
```

Using Joins to Enrich LLM Training Data

In LLM applications, you often need to combine text data with metadata, user profiles, or contextual information. Joins make this possible.

Start with Text Data

Your base DataFrame contains documents, conversations, or user inputs with unique IDs

Join User Context

Add user profiles, preferences, or history using left join on user_id

Add Metadata

Enrich with categories, timestamps, or quality scores from auxiliary tables

Create Rich Context

Your final dataset has all the context needed for effective fine-tuning or RAG

Performance Considerations

Tips for Efficient Joins

- **Merge on indexes for large joins:** set the join key as the index and use `left_index`/`right_index` (or `df.join`) — pandas optimizes index-based merges
- **Use categorical dtypes:** Convert string join keys to categorical type to reduce memory usage
- **Filter first:** Apply filters before joining to reduce the size of DataFrames being merged
- **Use appropriate join types:** Inner joins are fastest; outer joins are slowest
- **Index your keys:** Set join keys as DataFrame indexes for better performance

> **Memory Management:** Large joins can consume significant memory. Monitor memory usage with `df.memory_usage(deep=True)` and consider chunking very large datasets.

Key Takeaways

1

GroupBy = Aggregation Power

Use groupby().agg() to calculate multiple statistics across groups. Essential for feature engineering in AI pipelines.

2

Inner Join = Strict Matching

Returns only rows where keys exist in both DataFrames. Use when you need complete, validated data.

3

Left Join = Primary Preservation

Keeps all rows from the left DataFrame. Perfect for enriching your main dataset with optional supplementary data.

4

Outer Join = Complete Picture

Returns all rows from both sides. Useful for data quality checks and exploratory analysis.

5

Always Validate

Check row counts, inspect for NaNs, and verify key uniqueness. Validation prevents downstream pipeline errors.

Thanks